INSTITUTO TECNOLÓGICO DE AERONÁUTICA

João Matheus Del Vecchio

Do Dado Bruto ao Transformer

Manual didático passo a passo para compreender e defender o trabalho

São José dos Campos

2026

Como usar este manual

Este texto assume que o leitor começa sem domínio de microestrutura, processos pontuais ou redes neurais. Cada capítulo responde quatro perguntas: o que é, por que é necessário, como foi implementado e o que uma banca pode perguntar. Não memorize equações isoladas; aprenda a narrativa causal:

> mercado atualiza bid/ask → mid-price muda → surge evento → eventos se agrupam → Hawkes modela agrupamento → MLE estima parâmetros → rede aprende a imitar a estimação rapidamente → Transformer testa se atenção ajuda.

Sumário

1 A história inteira em linguagem simples **1**

2 Passo 1: compreender o mercado observado **2**

2.1 Bid, ask e spread . . . 2

2.2 Mid-price . . . 2

2.3 Por que bookTicker . . . 2

2.4 Por que futuros e não spot . . . 3

2.5 Perguntas da banca . . . 3

3 Passo 2: transformar milhões de linhas em eventos **4**

3.1 Por que 100 ms . . . 4

3.2 Algoritmo mental . . . 4

3.3 Interchegada . . . 4

3.4 Tick . . . 5

3.5 Por que chunks . . . 5

3.6 Contagens obtidas . . . 5

3.7 Perguntas da banca . . . 5

4 Passo 3: processos pontuais **6**

4.1 Do tempo regular ao tempo de eventos . . . 6

4.2 Contagem . . . 6

4.3 Intensidade . . . 6

5 Passo 4: Poisson e sua insuficiência **7**

6 Passo 5: processo de Hawkes **8**

6.1 Equação . . . 8

6.2 Self e cross . . . 8
6.3 Beta e memória . . . 9
6.4 Razão de ramificação . . . 9
6.5 Perguntas da banca . . . 9

7 Passo 6: simular Hawkes **10**
7.1 Por que simular . . . 10
7.2 Thinning de Ogata . . . 10

8 Passo 7: máxima verossimilhança **11**
8.1 Intuição . . . 11
8.2 Otimização . . . 11
8.3 Restrições . . . 11
8.4 Diagnóstico histórico, não final . . . 12

9 Passo 8: LSTM de Lee **13**
9.1 Entrada . . . 13
9.2 Memória recorrente . . . 13
9.3 Inferência amortizada . . . 13
9.4 O que é MSE . . . 13

10 Passo 9: Transformer **14**
10.1 Atenção . . . 14
10.2 Posição . . . 14
10.3 O caminho dos tensores . . . 14
10.4 Pooling . . . 15
10.5 Comparação justa . . . 15

11 Passo 10: interpretar os resultados sem exagerar **16**
11.1 Diagnósticos históricos e resultados finais . . . 16
11.2 O novo protocolo executável . . . 16
11.3 Frases corretas e incorretas . . . 17

12 Passo 11: reprodutibilidade **18**
12.1 Manifesto . . . 18

12.2 Parquet 18
12.3 Seeds 18
12.4 Git 18
12.5 Fonte única das métricas finais 19

13 Passo 12: como apresentar à banca **22**
13.1 Roteiro de quinze minutos 22
13.2 Perguntas difíceis 22

14 Passo 13: extensões após o protocolo final **24**

A Glossário **25**

B Comandos **26**

Capítulo 1

A história inteira em linguagem simples

Imagine que você observa uma fila. Pessoas não chegam uniformemente: às vezes ninguém chega; às vezes chega um grupo. Um processo de Poisson diria que cada chegada não muda a chance da próxima. Um processo de Hawkes diz que uma chegada pode aumentar temporariamente a chance de outra.

No mercado, o "cliente chegando" é uma alteração de preço. Uma alteração pode estimular outras porque algoritmos reagem, ordens são executadas, participantes atualizam expectativas e liquidez muda. Hawkes resume essa dinâmica por quatro números:

- μ: atividade que existiria sem eventos recentes;
- α_{self}: quanto uma alta estimula altas, ou baixa estimula baixas;
- α_{cross}: quanto alta estimula baixa, e vice-versa;
- β: quão rápido o efeito desaparece.

MLE encontra os números tentando várias combinações e escolhendo a que melhor explica os eventos. É relativamente lento. A proposta de Lee é simular muitos exemplos para os quais os números verdadeiros são conhecidos e treinar uma LSTM: dada uma sequência, ela cospe os quatro números em uma única passagem. Nosso trabalho reproduz isso em cripto e testa Transformer.

Capítulo 2

Passo 1: compreender o mercado observado

2.1 Bid, ask e spread

Bid é o maior preço que alguém oferece para comprar. **Ask** é o menor preço que alguém aceita para vender. Normalmente,

$$p_{\text{bid}} \leq p_{\text{ask}}.$$

A diferença é o spread. Exemplo: bid 41.734,90 e ask 41.735,00. Spread: 0,10.

2.2 Mid-price

O mid-price é o centro:

$$m = \frac{41.734{,}90 + 41.735{,}00}{2} = 41.734{,}95.$$

Ele não é necessariamente preço negociado. É uma medida do centro da melhor cotação, menos dependente de saber se o último negócio bateu no bid ou no ask.

2.3 Por que bookTicker

bookTicker registra alterações do melhor bid e ask. aggTrades registra negócios agregados. Para reproduzir o método, precisamos do primeiro. Trades servem como validação de

atividade, mas não recuperam com exatidão o melhor bid/ask.

2.4 Por que futuros e não spot

O acervo público histórico da Binance oferece bookTicker para futuros USD-M no período necessário. O caminho oficial de spot retornou ausência. Isso não significa que spot não tenha cotações ao vivo; significa que a fonte histórica gratuita escolhida não oferece o artefato necessário. A escolha é metodológica, não preferência comercial.

2.5 Perguntas da banca

Por que não usar último negócio? Porque ele alterna entre lados do livro e adiciona bid–ask bounce. Mid-price representa melhor o centro da cotação.

Por que Binance? Disponibilidade pública, granularidade e liquidez; não porque a Binance represente todo o mercado.

Por que perpétuos? Porque o acervo histórico oferece top-of-book para USD-M.

Isso cria viés? Sim. Resultados são condicionados à exchange e ao instrumento.

Capítulo 3

Passo 2: transformar milhões de linhas em eventos

3.1 Por que 100 ms

Dados chegam em milissegundos e contêm enorme atividade de atualização. Lee observa o mid-price em intervalos de 0,1 s para estudar alta frequência, não ultra-alta frequência. Em cada bin, usamos o último estado conhecido.

3.2 Algoritmo mental

1. pegue atualizações entre 12:00:00,000 e 12:00:00,099;
2. mantenha a última;
3. calcule o mid-price;
4. compare com o bin anterior;
5. se aumentou, evento 1; se diminuiu, evento 2; se igual, nada;
6. registre quanto tempo passou desde o evento anterior;
7. divida o tamanho da mudança pelo tick size.

3.3 Interchegada

Se eventos ocorreram em 3,8 s, 3,9 s e 4,1 s, interchegadas são 3,8 s a partir do início, depois 0,1 s e 0,2 s. Redes recebem interchegada porque ela descreve ritmo.

3.4 Tick

Tick é o menor incremento permitido. Se BTC tem tick 0,1 e mid-price muda 2,5, o salto tem 25 ticks. Usar ticks torna o tamanho comparável à estrutura do instrumento.

3.5 Por que chunks

Um dia pode ter 1,7 GB descompactado. Carregar tudo exigiria múltiplos gigabytes de RAM. O programa lê um milhão de linhas por vez. Guarda o último bin incompleto para combiná-lo com o próximo chunk; sem isso, uma fronteira artificial poderia criar estado errado.

3.6 Contagens obtidas

Tabela 3.1: Resultado da transformação principal.

Símbolo	Eventos	Janelas de 1.000 eventos
BTCUSDT	887.803	880
ETHUSDT	939.101	933

3.7 Perguntas da banca

Por que manter a última atualização? Ela representa o estado observável no fim do intervalo.

Por que eliminar preços iguais? O modelo representa eventos de mudança.

Por que aparecem timestamps repetidos? Várias atualizações podem compartilhar resolução temporal; reduzimos ao último estado antes da grade.

O filtro perde informação? Sim, deliberadamente. Ele reduz ruído e alinha o estudo à escala do paper.

Capítulo 4

Passo 3: processos pontuais

4.1 Do tempo regular ao tempo de eventos

Uma série comum registra preço a cada segundo. Processo pontual registra os próprios instantes em que algo aconteceu. A sequência é

$$(t_1, c_1), (t_2, c_2), \ldots,$$

em que $c_k = 1$ ou 2.

4.2 Contagem

$$N_i(t) = \text{número de eventos do tipo } i \text{ até } t.$$

Se ocorreram três altas até 10 s, $N_1(10) = 3$.

4.3 Intensidade

Intensidade é taxa condicional. Se $\lambda_1(t) = 5$ eventos/s, em intervalo pequeno de 0,01 s esperamos aproximadamente 0,05 eventos de tipo 1. Não é dizer que "ocorrerá 0,05 evento"; é uma média/probabilidade infinitesimal.

Capítulo 5

Passo 4: Poisson e sua insuficiência

No Poisson homogêneo:

$$\lambda(t) = \lambda_0.$$

O passado não importa. Interchegadas são exponenciais. Se dados exibem excesso de intervalos curtos e rajadas, Poisson subestima agrupamento. Hawkes permite que cada evento aumente a taxa futura.

Capítulo 6

Passo 5: processo de Hawkes

6.1 Equação

$$\lambda_i(t) = \mu + \sum_j \sum_{t_k^j < t} \alpha_{ij} e^{-\beta(t - t_k^j)}.$$

Leia da esquerda para direita:

1. intensidade atual do tipo i;
2. começa em μ;
3. soma influência de todos os eventos passados;
4. influência inicial é α_{ij};
5. influência decai exponencialmente com distância temporal.

6.2 Self e cross

Na matriz simétrica,

$$\alpha = \begin{bmatrix} \alpha_s & \alpha_c \\ \\ \alpha_c & \alpha_s \end{bmatrix}.$$

α_s mede persistência direcional do tipo de evento. α_c mede reversão ou reação entre tipos. A interpretação econômica deve ser cuidadosa: parâmetro não prova causalidade comportamental.

6.3 Beta e memória

A meia-vida aproximada do kernel é

$$t_{1/2} = \frac{\log 2}{\beta}.$$

Beta grande: memória curta. Beta pequeno: efeito persiste.

6.4 Razão de ramificação

$$g = \frac{\alpha_s + \alpha_c}{\beta}.$$

Se $g = 0,8$, cada evento produz, em média, 0,8 filhos diretos. Filhos produzem netos; o cluster esperado cresce, mas permanece finito. Se $g \geq 1$, a cascata pode não morrer.

6.5 Perguntas da banca

Por que kernel exponencial? É parcimonioso, possui recursão eficiente e segue o artigo-base.

Por que simétrico? Reduz dimensão e reproduz Lee; não afirma que o mercado é perfeitamente simétrico.

Por que $g < 1$? Para estabilidade/estacionariedade.

Por que cross quase zero no MLE atual? É indício, não conclusão; pode ser dado, identificação, otimização ou especificação e requer diagnóstico.

Capítulo 7

Passo 6: simular Hawkes

7.1 Por que simular

Em dados reais, os parâmetros verdadeiros são desconhecidos. Em simulação, escolhemos θ, geramos sequência e perguntamos se o estimador recupera θ. Isso cria treino supervisionado sem rótulo humano.

7.2 Thinning de Ogata

1. calcule um limite superior M para intensidade total;
2. sorteie espera $w \sim \text{Exp}(M)$;
3. avance tempo e deixe excitações decaírem;
4. calcule intensidade real λ_{tot};
5. aceite com probabilidade λ_{tot}/M;
6. se aceito, sorteie tipo proporcionalmente a λ_1, λ_2;
7. adicione excitação e repita.

O algoritmo rejeita alguns candidatos, exatamente como selecionar pontos de um processo mais intenso para produzir o processo desejado [1].

Capítulo 8

Passo 7: máxima verossimilhança

8.1 Intuição

MLE procura parâmetros que tornem o observado plausível. A função é

$$\ell = \underbrace{\sum_k \log \lambda_{c_k}(t_k)}_{\text{recompensa eventos}} - \underbrace{\sum_i \int_0^T \lambda_i(u)\, du}_{\text{penaliza intensidade excessiva}} .$$

Sem compensador, bastaria elevar todas as intensidades.

8.2 Otimização

L-BFGS-B procura máximo numericamente. Como pode depender da inicialização, usamos múltiplos pontos. O código converte problema em minimização de $-\ell$.

8.3 Restrições

$$\mu = \text{softplus}(z_1), \quad \alpha_s = \text{softplus}(z_2), \quad \alpha_c = \text{softplus}(z_3), \quad d = \text{softplus}(z_4),$$

$$\beta = \alpha_s + \alpha_c + d.$$

Softplus, $\log(1 + e^z)$, é sempre positiva e diferenciável.

8.4 Diagnóstico histórico, não final

Em uma verificação inicial, doze ajustes representativos convergiram. Esse registro histórico apenas confirma que o pipeline executou com janelas de 1.000 eventos; ele não constitui medição final de erro ou desempenho. As métricas finais vêm exclusivamente de `thesis/latex/final_results.tex`.

Capítulo 9

Passo 8: LSTM de Lee

9.1 Entrada

Cada passo tem dois números: $\log(1+\Delta t)$ e tipo codificado como $+1/-1$. O log comprime interchegadas muito grandes.

9.2 Memória recorrente

A LSTM lê um passo, atualiza memória, lê o seguinte e assim por diante. Duas camadas com 12 unidades produzem estado final. Uma camada linear produz quatro logits, depois transformados em parâmetros válidos.

9.3 Inferência amortizada

MLE otimiza novamente para cada janela. LSTM treina uma vez, depois apenas executa multiplicações. Treino pode ser caro; inferência tende a ser barata. "Mais rápida" deve ser comparado no mesmo hardware, lote e protocolo.

9.4 O que é MSE

$$\mathrm{MSE} = \frac{1}{n}\sum_{r=1}^{n} \|\hat{\theta}_r - \theta_r\|^2.$$

Erros grandes pesam quadraticamente. Também precisamos métricas por parâmetro porque escalas diferentes podem dominar a média.

Capítulo 10

Passo 9: Transformer

10.1 Atenção

Cada evento cria query, key e value. Produto query–key indica relevância. Softmax transforma escores em pesos. Value combina informação. Múltiplas cabeças podem aprender padrões diferentes.

10.2 Posição

Sem encoding, Transformer não sabe ordem. Adicionamos senoides dependentes da posição. Uma extensão futura pode usar encoding do tempo acumulado, pois eventos são irregulares.

10.3 O caminho dos tensores

Começamos com uma caixa de formato $[B, L, 2]$: B sequências, L eventos e dois atributos por evento. Uma camada linear transforma cada par em 32 números, dando $[B, L, 32]$. Somamos posição. Quatro cabeças recebem oito números por posição cada uma e aprendem relações diferentes. O bloco feed-forward expande cada posição de 32 para 128 números e volta para 32; isso ocorre duas vezes, pois há duas camadas encoder.

Não são quatro redes independentes: as cabeças são partes paralelas da mesma camada. Query pergunta “o que procuro?”, key descreve “o que ofereço?” e value contém “qual informação entrego?”. O produto query–key, seguido de softmax, decide quanto de cada value será misturado na posição atual.

10.4 Pooling

Encoder produz vetor por posição. A média resume sequência em um vetor, então a mesma cabeça estável gera parâmetros.

A média muda $[B, L, 32]$ para $[B, 32]$. A cabeça produz quatro números positivos: $\mu, \alpha_s, \alpha_c, d$. O quarto parâmetro apresentado é construído como $\beta = \alpha_s + \alpha_c + d$. Assim $(\alpha_s + \alpha_c)/\beta < 1$ automaticamente; não dependemos de consertar uma previsão instável depois.

10.5 Comparação justa

Transformer e LSTM devem compartilhar dataset, splits, targets, loss e orçamento de busca. Caso contrário, não sabemos se ganho veio da arquitetura ou de tratamento preferencial.

Capítulo 11

Passo 10: interpretar os resultados sem exagerar

11.1 Diagnósticos históricos e resultados finais

Os ensaios reduzidos usados durante o desenvolvimento eram apenas verificações de software. Seus números estão obsoletos e não devem ser usados como evidência científica. As métricas finais sintéticas e empíricas, inclusive erro e latência, são reportadas somente pelo fragmento `thesis/latex/final_results.tex`, incorporado neste manual no Passo 11.

11.2 O novo protocolo executável

O protocolo atual usa exclusivamente sequências de 1.000 eventos. A configuração local usa 48/16/16 trajetórias e duas épocas apenas para encontrar erros de software. A configuração final usa 75.000 trajetórias para treino, 10.000 para validação, 15.000 para teste, até 300 épocas e cinco seeds. O comando `make smoke` valida o pipeline, mas não produz métricas científicas finais; os comandos finais são separados porque podem consumir horas ou dias.

Nos dados reais, todas as janelas também contêm 1.000 eventos. O treinamento só pode enxergar 15–22 de janeiro. A escolha usa 23–25. O teste 26–28 exige abertura explícita. Outubro usa o checkpoint congelado, sem treinar novamente. Se a banca perguntar por que, a resposta é: permitir que o modelo aprenda com o teste transforma medição de generalização em memorização.

11.3 Frases corretas e incorretas

Correta: “O protocolo final compara os modelos em sequências de 1.000 eventos.”

Incorreta: “Há também uma execução final com outro comprimento de sequência.”

Correta: “As métricas finais vêm de `thesis/latex/final_results.tex`.”

Incorreta: “Os números das verificações de software são resultados finais.”

Correta: “A fonte escolhida cobre as janelas estudadas.”

Incorreta: “Os dados representam todo o mercado de cripto.”

Capítulo 12

Passo 11: reprodutibilidade

12.1 Manifesto

Cada ZIP recebe URL, tamanho e SHA-256. SHA-256 funciona como impressão digital: se um byte mudar, hash muda com altíssima probabilidade.

12.2 Parquet

CSV é volumoso e sem tipos fortes. Parquet é colunar, comprimido e eficiente para ler somente colunas necessárias. O CSV temporário é apagado após conversão.

12.3 Seeds

Seed torna sorteio reproduzível. Não elimina aleatoriedade científica; permite repetir uma realização. Resultado final precisa de várias seeds.

12.4 Git

Código e configurações entram no Git; dados pesados, checkpoints e artefatos não. O manifesto permite reconstruí-los. Um commit identifica exatamente a versão do código.

12.5 Fonte única das métricas finais

As métricas finais não são digitadas manualmente neste manual. Elas são geradas em `thesis/latex/final_results.tex` a partir do resumo consolidado e incorporadas abaixo.

Resultados da execução final otimizada

O protocolo final usa exclusivamente trajetórias de 1.000 eventos, com 75.000 amostras de treino, 10.000 de validação e 15.000 de teste. As redes são resumidas sobre cinco seeds independentes; o MLE é ajustado separadamente em cada trajetória do mesmo conjunto de teste. Os intervalos das redes usam a distribuição t de Student entre seeds.

Tabela 12.1: Métricas agregadas no teste sintético congelado (15.000 trajetórias de 1.000 eventos).

Estimador	MSE macro	IC 95%	MAE macro	MSE de g	Validade
LSTM	0,019618	[0,013252; 0,025984]	0,094276	0,001825	100,00%
Transformer	0,023700	[0,019128; 0,028272]	0,105642	0,002138	100,00%
MLE	0,506193	–	0,111852	0,002344	100,00%

Tabela 12.2: MSE de teste por parâmetro (redes: média de cinco seeds).

Estimador	μ	α_{self}	α_{cross}	β
LSTM	0,002296	0,016222	0,016566	0,043386
Transformer	0,002366	0,020905	0,021196	0,050332
MLE	0,010108	0,006146	0,006214	2,002303

Tabela 12.3: MSE macro por regime da razão de ramificação g.

Estimador	g baixo	g intermediário	g quase crítico
LSTM	0,019941	0,015614	0,023297
Transformer	0,022228	0,018964	0,029908
MLE	1,480568	0,016795	0,021214

Entre as redes, a LSTM obteve MSE macro médio 17,2% menor que o Transformer. Com 5 e 5 seeds, porém, a diferença não é estatisticamente significativa (teste de Welch, $p = 0,19$), e os intervalos de confiança se sobrepõem; os dados não permitem ordenar as

redes. A comparação deve considerar conjuntamente erro, estabilidade entre seeds e custo de inferência, e não apenas a melhor seed isolada.

No MLE, o parâmetro β respondeu por 98,9% da soma dos erros quadráticos, e o MSE macro no regime de g baixo foi 88,15 vezes o do regime intermediário: com excitação fraca, β é pouco identificado e algumas trajetórias produzem estimativas extremas. Fora desse regime, o MLE é competitivo: no regime intermediário, seu MSE macro foi 0,0168 (LSTM: 0,0156; Transformer: 0,0190) e, no quase crítico, 0,0212 (LSTM: 0,0233; Transformer: 0,0299). O MLE obteve o menor MSE entre os três estimadores em α_{self} e α_{cross}. Portanto, o MSE macro agregado não ordena os estimadores de forma robusta, e convergência numérica não implica boa recuperação paramétrica.

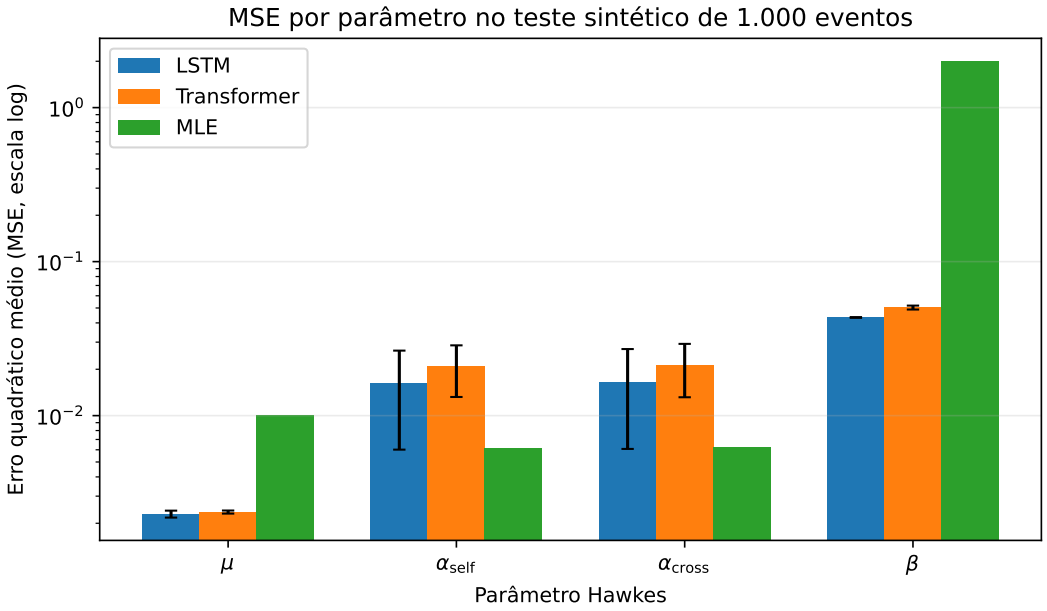


Figura 12.1: MSE de teste por parâmetro no protocolo de 1.000 eventos. As barras de erro das redes representam um desvio-padrão entre seeds.

O MLE convergiu em 100,00% das trajetórias e usou mediana de 14,00 iterações. Todas as comparações usam o mesmo conjunto de teste congelado.

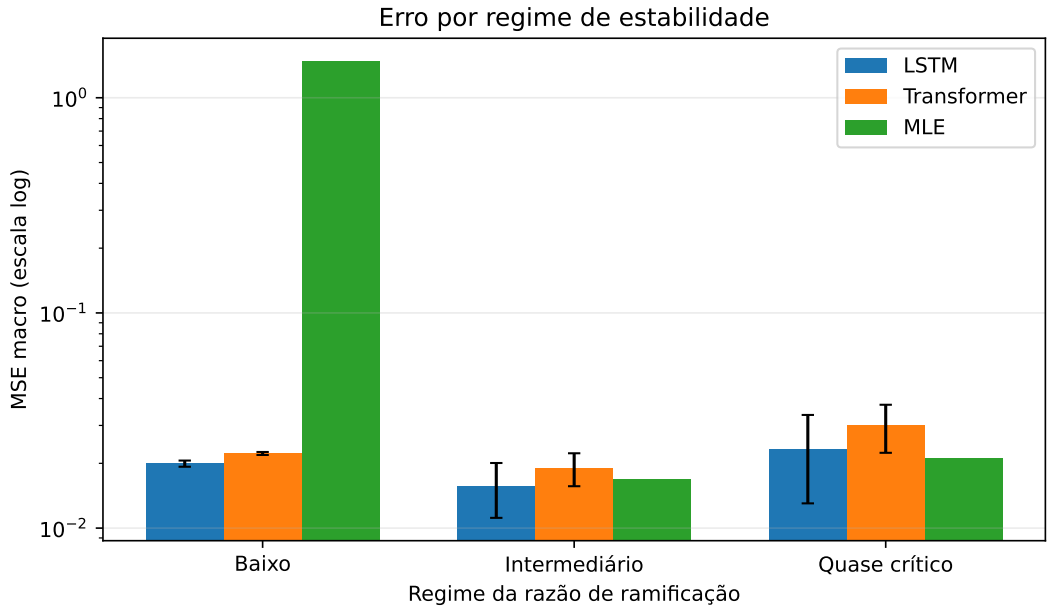


Figura 12.2: MSE macro por regime da razão de ramificação. As barras de erro das redes representam um desvio-padrão entre seeds.

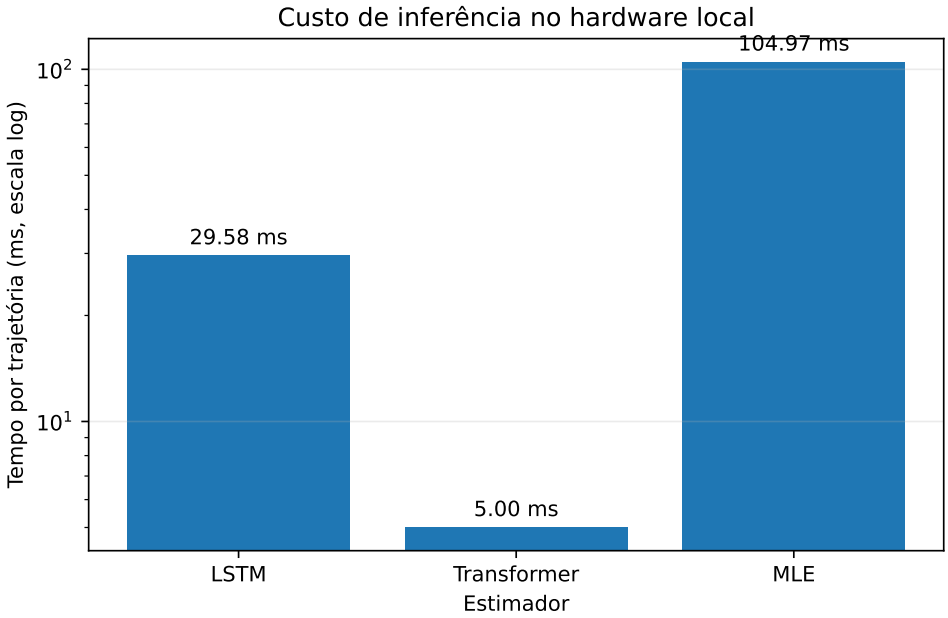


Figura 12.3: Custo por trajetória registrado durante a execução final, em escala logarítmica. Para o MLE, tempo de parede com quatro processos que concorriam com o treino na GPU; para as redes, latência mediana em batch unitário no MPS. Um benchmark controlado posterior mediu cerca de 10 ms por trajetória para o MLE em um único núcleo e deve ser preferido a esta figura.

Capítulo 13

Passo 12: como apresentar à banca

13.1 Roteiro de quinze minutos

1. 1 min: problema—clustering de mudanças de preço;
2. 2 min: Hawkes e interpretação dos quatro parâmetros;
3. 2 min: gargalo do MLE e ideia de Lee;
4. 2 min: gap—cripto e atenção;
5. 2 min: dados e transformação;
6. 2 min: desenho experimental sem vazamento;
7. 2 min: métricas finais do fragmento gerado;
8. 1 min: limitações;
9. 1 min: próximos passos e contribuição.

13.2 Perguntas difíceis

Por que usar MLE como rótulo real? Porque parâmetro verdadeiro não existe em dados reais; imitar MLE é objetivo operacional. Mas herda seus vieses.

Por que não Neural Hawkes? Porque o objetivo é estimar parâmetros interpretáveis do Hawkes clássico, não substituir sua intensidade por caixa-preta.

Como evitar vazamento? Split por blocos temporais antes de gerar janelas.

Por que duas semanas? Volume já gera milhões de eventos e mantém execução administrável; robustez é testada em semana separada.

Por que usar apenas 1.000 eventos? Porque esse é o comprimento único fixado pelo

protocolo vigente. Resultados com outros comprimentos não integram a execução final nem as comparações reportadas.

Qual contribuição se Transformer perder? Reprodução, dataset/pipeline pública e evidência controlada de que atenção não traz ganho nesse regime.

Capítulo 14

Passo 13: extensões após o protocolo final

1. ampliar o MLE empírico nas janelas de 1.000 eventos;
2. abrir o teste empírico fechado uma única vez;
3. avaliar outubro sem tuning nem retreinamento;
4. diagnosticar resíduos do Hawkes;
5. calcular e comparar volatilidades;
6. revisar escrita, referências e normas ITA.

Apêndice A

Glossário

Ask: menor preço ofertado por vendedor.

Bid: maior preço ofertado por comprador.

Evento: mudança de mid-price após filtro.

Inferência amortizada: custo de treino reutilizado em inferências futuras.

Intensidade: taxa condicional instantânea de eventos.

Kernel: forma temporal do efeito de um evento.

MLE: estimativa por máxima verossimilhança.

Mid-price: média entre bid e ask.

Self-attention: mecanismo que relaciona posições da mesma sequência.

Split: partição de treino, validação ou teste.

Tick: menor incremento de preço permitido.

Apêndice B

Comandos

```
uv sync --python 3.13 --extra dev
uv run python scripts/build_primary_dataset.py --workers 4
uv run python scripts/index_event_windows.py
make smoke
make synthetic-final
make train-final
make calibration
make empirical-test
uv run python scripts/generate_final_report_fragment.py
uv run ruff check .
uv run mypy src/hawkes_crypto
uv run pytest
```

Referências Bibliográficas

[1] Yosihiko Ogata. On Lewis' simulation method for point processes. *IEEE Transactions on Information Theory*, 27(1):23–31, 1981. doi: 10.1109/TIT.1981.1056305.